校园互助服务平台（CampusHub）- AI 工具选型评估表

文档信息

- **所属阶段：** Phase 0 项目启动与 AI 协作契约
- **编写负责人：** 241250015 徐航宇
- **团队名称：** BUG 一改就队
- **评估日期：** 2026 年 4 月 11 日

评估背景与目标

为确保 CampusHub 平台在 10 周内高效完成从需求分析到终期交付的全生命周期开发，团队对市面主流的 AI 工具进行了调研与评估。结合团队的技术栈熟悉度与实际需求，我们从“AI 编程助手”、“AI 对话工具”、“AI 代码审查”及“AI 测试工具”四个维度进行了严谨的对比，最终确立了本项目的 AI 工具链。

一、AI 编程助手（辅助代码生成、补全、重构）

候选工具评估

评估维度	候选工具 A：GitHub Copilot	候选工具 B：Cursor (IDE)	候选工具 C：通义灵码
功能特性	行业标杆。提供极高质量的上下文代码补全、Chat 问答、多文件关联理解。	基于 VS Code 深度定制的 AI 编辑器，支持全局代码库检索与重写（Composer）。	阿里出品，国内响应极快，擅长企业级业务代码生成及中文注释撰写。
易用性	高。作为插件无缝集成到团队成员现有的 VS Code / IDEA 中。	中。需要全员迁移到新的编辑器软件，存在一定的适应成本。	极高。插件安装即用，国内网络环境极度稳定。
成本	付费（$10/月），但**学生可通过 GitHub Student Pack 免费获取**。	免费版有严格的高级模型使用次数限制。	完全免费。
团队熟悉度	高（团队成员有使用经验或已完成学生认证）。	较高。	中等。

◎ 最终选型决策

- **选择工具：【GitHub Copilot】**
- **选择理由：**作为老牌 AI 编程助手，Copilot 的上下文预测能力和跨文件理解能力在业界无可挑剔。团队成员已具备相关使用经验（或已开通学生包），直接集成到现有 IDE 中能实现**“零迁移成本”**，在 P4 核心编码阶段能最大化保障开发节奏。
- **不选择其他工具的理由：**
  - 不选 Cursor：虽然全局生成能力强，但强制要求团队更换 IDE 会增加不必要的协同摩擦。
  - 不选通义灵码：在处理复杂业务逻辑（如模块四订单状态机流转）时，其上下文理解深度略逊于 Copilot。

二、AI 对话工具（辅助文档生成、需求分析、架构设计）

候选工具评估

评估维度	**候选工具 A：Claude (Sonnet / Opus 4.6)**	**候选工具 B：ChatGPT (GPT-5.4)**	**候选工具 C：DeepSeek (V3/R1)**
功能特性	**软件工程领域的当前最优模型之一**。极少“偷懒”，代码与文档输出完整，极其擅长处理长篇架构文档和重构逻辑。	全面且强大。逻辑推理顶级，但在生成长段代码或超长文档时偶尔会出现省略号（偷懒）。	国产头部大模型。深度思考能力极强，上下文窗口极大。
易用性	较高。Artifacts 侧边栏功能非常适合预览前端页面或结构化图表（如类图代码）。	高。多模态能力强，交互体验成熟。	极高。国内直连，无需复杂网络配置。
成本	基础使用免费，可白嫖高质额度。	免费版有频次限制。	完全免费。
团队熟悉度	较高（团队主力生产力工具）。	较高。	较高。

◎ 最终选型决策

- **选择工具：【Claude】**
- **选择理由：**在 P1-P3（需求到详细设计）阶段，我们需要生成大量如《需求规格说明书》、ADR（架构决策记录）等专业文档。Claude 的 **Artifacts 功能**和**不折不扣的输出完整度**远超同行，其生成的代码质量和文档结构严谨度完美契合软件工程课程的标准。

- **不选择其他工具的理由：**
  - 不选 ChatGPT：在要求输出完整系统类图或长篇文档时，经常使用“... 这里省略...”来敷衍，需要反复 Prompt 纠正，浪费时间。
  - 不选 DeepSeek：虽然免费且强大，但 Claude 在代码实现细节和排版格式（Markdown 表现力）上更符合我们的工程审美。

三、AI 代码审查工具（保障代码合入质量）

候选工具评估

评估维度	候选工具 A：CodeRabbit	候选工具 B：AI Code Reviewer (Action)	候选工具 C：SonarQube (结合 AI)
功能特性	**专为 PR/MR 打造的 AI 审查平台**。自动总结修改内容、逐行点评代码缺陷、识别潜在漏洞，并支持对话。	基于 GitHub Actions 的轻量级开源脚本，调用 API 进行基础的代码 Diff 审查。	传统静态代码扫描工具结合 AI 扩展，侧重于技术债和规范检查。
易用性	极高。在 GitHub 安装 App 即可，全自动触发，零维护成本。	较低。需自行配置 YML 流水线并维护 API Key。	最低。需自行搭建服务器和配置规则集。
成本	对 Public（公开）仓库完全免费。	需要消耗个人 API Token 额度（可能产生费用）。	部署与维护成本极高。
团队熟悉度	中（需简单了解工作流）。	低。	低。

◎ 最终选型决策

- **选择工具：【CodeRabbit】**
- **选择理由：**《AI 协作契约》严格规定“生成代码必须经过审查”。CodeRabbit 能无缝嵌入 GitHub PR 流程，充当不知疲倦的“资深工程师”，不仅能帮团队揪出潜在 Bug（如非功能需求中的 XSS 漏洞），还能自动生成高质量的 PR 摘要，完美覆盖 P6 质量保障阶段的需求。
- **不选择其他工具的理由：**
  - 不选开源 Action / SonarQube：配置繁琐，对学生团队 10 周的敏捷开发周期来说太重了，且开源脚本无法像 CodeRabbit 那样在评论区进行多轮上下文对话纠错。

四、AI 测试工具（测试用例设计与生成）

候选工具评估

评估维度	候选工具 A：Claude	候选工具 B：GitHub Copilot	候选工具 C：CodiumAI
功能特性	将核心代码输入对话框，能深度分析边界条件，生成覆盖率极高的单元测试代码。	通过 /tests 命令在 IDE 内快速生成关联文件的基础测试用例。	专门针对测试生成的 IDE 插件，能可视化分析代码并推荐测试集。
易用性	高。直接复制粘贴代码并提供 Prompt 即可。	极高。一键生成。	中。需要学习额外的插件界面和操作逻辑。
覆盖深度	**深**。擅长挖掘复杂状态机（如订单流转）的异常分支。	较浅。通常只覆盖常规的 Happy Path（主流程）。	较深。

◎ 最终选型决策

- **选择工具：【Claude】**
- **选择理由：** P4 阶段要求“关键模块有单元测试”。我们发现，通过精心设计的 Prompt，将核心类库（如智能匹配模块、订单状态机）投喂给 Claude，它能比 IDE 插件更敏锐地识别出**边界条件**和**异常路径**，从而生成高质量的单元测试，这比单纯追求快速生成的测试更具工程价值。
- **不选择其他工具的理由：**
  - 不选 Copilot /tests：测试用例往往停留在表面，缺乏对业务逻辑深层次破坏性测试的考量。
  - 不选 CodiumAI：团队希望保持工具链的精简，避免安装过多的单一功能插件增加认知负担，Claude 已经能出色完成此任务。

总结：CampusHub 团队 AI 工具链全景图

经过严密评估，本团队最终确定的 AI 协同开发基座如下：

- **核心生产力：** **GitHub Copilot**（编码提效）+ **Claude**（架构设计/文档生成/测试用例）
- **质量守门员：** **CodeRabbit**（自动化 PR 审查拦截缺陷）